



دانشکده مهندسی کامپیوتر  
سیستم های عامل

نیمسال اول ۱۴۰۵-۱۴۰۴  
۴۰۲۱۱۰۸۷۶، ۴۰۲۱۰۶۵۴۲

حسین اسدی  
علی مقدسی، علیرضا سرباز

تمرین دوم

## ۱ سوال ۱

### ۱.۱ آ

- **بهبود اشکال زدایی:** مرزهای شفاف بین لایه ها، ردگیری خطا را ساده می کند؛ برای نمونه در لینوکس، اگر فراخوانی فایل شکست بخورد، می توان مسیر خطا را از VFS به لایه سیستم فایل خاص (مثل ext4) و سپس به لایه بلوک دنبال کرد. قراردادهای ثابت واسطه ها، محل نقص را محدود می کنند.
  - **مانع اشکال زدایی:** وقتی رفتار از مرز لایه ها عبور می کند (Caching، تاخیرها، نوشتن تنبل)، اثرات جانبی در لایه ای ظاهر می شود که علت در لایه دیگر است؛ مثلاً ناسازگاری inode و dentry در VFS ممکن است به صورت خطای ENOENT در سطح glibc بروز کند.
  - **بهبود گسترش پذیری:** API های پایدار، افزودن پیاده سازی های جدید را آسان می کنند، VFS اجازه می دهد بدون تغییر برنامه ها، سیستم فایل های جدید (F2FS، btrfs) افزوده شوند. مشابه همین، پشته I/O شبکه با socket API امکان افزودن TLS offload یا XDP را می دهد.
  - **مانع گسترش پذیری:** اگر API لایه بالاتر قابلیت جدیدی را مدل نکند، باید شکستن API رخ دهد؛ مثلاً ویژگی های خاص NVMe ZNS یا io\_uring ابتدا خارج از مسیر سنتی POSIX افزوده شدند تا از قیود read/write عبور کنند.
  - **کارایی بهتر:** جداسازی concerns اجازه می دهد هر لایه بهینه سازی محلی داشته باشد؛ page cache و dentry cache در VFS نگاه به عقب های گران مسیر را کم می کنند؛ NAPI در لایه شبکه نرخ وقفه را کنترل می کند.
  - **کاهش کارایی:** هر پرش لایه ای، انتساب ساختارها و کپی داده ها هزینه دارد؛ مسیر کامل read() از glibc تا VFS، سیستم فایل، لایه بلوک و درایور می تواند چندین بار بررسی دسترسی و نگاشت ساختار انجام دهد. مکانیزم هایی مانند splice()، O\_DIRECT، و io\_uring برای دورزدن بخشی از لایه ها و کاهش سربار معرفی شدند.
- نمونه های واقعی: در لینوکس، VFS مزیت توسعه پذیری و اشکال زدایی را بهبود داده، اما برای کارایی، مسیرهای fast-path مثل page cache hit یا direct I/O طراحی شده اند تا از هزینه لایه ها بکاهند. در پشته I/O شبکه، XDP بسته ها را پیش از ساخت sk\_buff در درایور پردازش می کند.

### ۲.۱ ب

هسته های مبتنی بر مازول (مانند مازول های قابل بارگذاری لینوکس) با طراحی صرفاً لایه ای تفاوت های کلیدی دارند:

- **جهت جریان:** در طراحی لایه‌ای، جریان فراخوانی عمده‌تاً یک‌سویه و نزولی است. در معماری ماژولی، مؤلفه‌ها از طریق ثبت‌نام واسطه‌ها، callback ها و hook ها به‌صورت پویا به هم متصل می‌شوند (مثلاً VFS inode/file LSM، netfilter، ops).
- **انعطاف‌پذیری زمان‌اجرا:** ماژول‌ها می‌توانند load/unload شوند، قابلیت جدید بیافزایند یا درایور تعویض کنند بدون reboot. این انعطاف در طراحی صرفاً لایه‌ای ایستا مشکل است.
- **ایزوله‌سازی:** مرز ماژولی تا حدی ایزولاسیون سازمانی/مهندسی ایجاد می‌کند (نمادهای صادرشده، refcount، مالکیت منابع). بااین‌حال، چون همه در فضای هسته‌اند، ایزوله‌سازی حافظه‌ای قوی شبیه فرآیندهای کاربر ندارند؛ یک oops در ماژول می‌تواند کل سیستم را تحت تاثیر قرار دهد. مزیت عملی: امکان حذف یا جایگزینی ماژول معیوب و آزادسازی منابع بدون خاموشی کامل.
- **گسترش‌پذیری:** افزونه‌های امنیتی (eBPF/LSM)، شتاب‌دهنده‌های crypto، فایل سیستم‌ها و درایورها به صورت افزونه بارگذاری می‌شوند؛ این از قیود سفت لایه‌ای عبور می‌کند بدون شکستن قراردادهای عمومی.

## ۳.۱ ج

- در مسیرهای حساس به کارایی، قوانین لایه‌بندی عمده‌تاً نادیده گرفته می‌شوند تا overhead کاهش یابد:
- **شبکه:** XDP و برخی مسیرهای eBPF بسته‌ها را پیش از ورود به پشته کامل شبکه پردازش می‌کنند؛ GRO/TSO نیز حدود لایه‌بندی را جابه‌جا می‌کنند. نتیجه: تاخیر کمتر و نرخ بسته بالاتر.
  - **فایل/دیسک:** O\_DIRECT و io\_uring بخشی از VFS/page cache را دور می‌زنند تا کپی و قفل‌های غیرضروری حذف شود. مسیرهای bio گاهی مستقیم‌تر به درایور می‌رسند.
  - **زمان‌بندی:** مسیرهای fast-path زمان‌بند با ساختارهای per-CPU و دسترسی مستقیم به صف‌های آماده کار می‌کنند تا قفل‌گیری‌ها و فراخوانی‌های لایه‌ای کم شود.
  - **خطرات:** نقض لایه‌بندی می‌تواند موجب نشت انتزاع، تکرار منطق اعتبارسنجی، دورزدن سیاست‌های امنیتی (LSM hooks)، شکنندگی API ها، سخت‌تر شدن اشکال‌زدایی و افزایش ریسک خطاهای رقابتی شود. بنابراین معمولاً این مسیرها با قراردادهای دقیق، ممیزی و تست سنگین محافظت می‌شوند.

## ۲ سوال ۲: Bomb Fork

یکی از مواردی که می‌تواند سیستم را دچار مشکل کند ایجاد Fork به صورت انفجاری است. در این تمرین قصد داریم به بررسی مشکلاتی که Fork Bomb ایجاد می‌کند و نحوه رفع آن بپردازیم.

### ۱.۲ مفهوم Bomb Fork و نحوه عملکرد آن

Bomb Fork یک نوع حمله Service of Denial است که با ایجاد تعداد بسیار زیادی پردازش در مدت زمان کوتاه، منابع سیستم (CPU، حافظه، و جدول پردازش‌ها) را مصرف می‌کند و سیستم را غیرقابل استفاده می‌سازد.

نحوه عملکرد Bomb Fork معمولاً یک حلقه بی‌نهایت است که در هر تکرار، با استفاده از فراخوان سیستمی fork()، یک کپی از خودش ایجاد می‌کند. هر پردازش جدید نیز همین کار را انجام می‌دهد، بنابراین تعداد پردازش‌ها به صورت نمایی رشد می‌کند. برای مثال، اگر در هر ثانیه هر پردازش یک پردازش جدید ایجاد کند، پس از ۱۰ ثانیه بیش از ۱۰۰۰ پردازش خواهیم داشت، و پس از ۲۰ ثانیه بیش از یک میلیون پردازش.

مشکلات ایجاد شده:

- **مصرف CPU:** هر پردازش جدید نیاز به زمان CPU دارد و زمان‌بند سیستم برای مدیریت این تعداد زیاد پردازش دچار مشکل می‌شود.
- **مصرف حافظه:** هر پردازش نیاز به فضای حافظه دارد (حتی اگر کوچک باشد) و با تعداد زیاد پردازش‌ها، حافظه سیستم به سرعت پر می‌شود.
- **پر شدن جدول پردازش‌ها:** سیستم‌عامل برای هر پردازش یک ورودی در جدول پردازش‌ها نگه می‌دارد. این جدول محدود است و با پر شدن آن، سیستم نمی‌تواند پردازش جدیدی ایجاد کند.
- **مشکل در زمان‌بندی:** زمان‌بند سیستم برای انتخاب پردازش بعدی باید از میان تعداد بسیار زیادی پردازش انتخاب کند که باعث کاهش کارایی می‌شود.

## ۲.۲ نمونه Bomb Fork ساده

یک نمونه ساده Bomb Fork در فایل fork\_bomb.c پیاده‌سازی شده است:

```
<h. dtsinu> edulcni#
<h. oidts> edulcni#
<h. sepyt / sys> edulcni#

    } () niam tni
    } (۱) elihw
    ;() krof
    {
        ;۰ nruter
    }
```

برای کامپایل و اجرا:

```
fork_bomb.c fork_bomb -o gcc
./fork_bomb
```

⚠ **هشدار:** این برنامه سیستم را دچار مشکل می‌کند! حتماً در ماشین مجازی اجرا کنید.  
نتایج اجرا: RESULT\_TODO

## ۳.۲ اثرات Bomb Fork روی منابع سیستم

برای تحلیل اثرات Bomb Fork روی منابع سیستم، می‌توان از دستورات زیر استفاده کرد:

```
processes of number View #
-l wc | aux ps
```

```
usage memory and CPU View #
top
or #
htop
```

```
limits system View #
-a ulimit
```

آیا مشکل با یک Ctrl+C ساده حل می‌شود؟ چرا؟  
RESULT\_TODO  
توضیح: RESULT\_TODO

## ۴.۲ راه‌های مقابله با Bomb Fork

راه‌های مختلفی برای مقابله با Bomb Fork وجود دارد:

۱. استفاده از **ulimit**: می‌توان تعداد پردازش‌های مجاز برای یک کاربر را محدود کرد:

```
processes 100 to Limit # 100 -u ulimit
```

۲. استفاده از **cgroups**: در سیستم‌های مدرن لینوکس، می‌توان از **cgroups** برای محدود کردن منابع استفاده کرد.

۳. کشتن پردازش‌ها: استفاده از **killall** یا **pkill** برای متوقف کردن تمام پردازش‌های Bomb Fork:

```
fork_bomb 9- pkill  
or #  
fork_bomb 9- killall
```

۴. استفاده از اسکریپت خودکار: اسکریپت **stop\_fork\_bomb.sh** برای این منظور تهیه شده است.

۵. تنظیمات سیستم: می‌توان در فایل **/etc/security/limits.conf** محدودیت‌های دائمی تعریف کرد.

## ۵.۲ اجرای Bomb Fork و مهار آن

مراحل انجام:

۱. اجرای Bomb Fork در یک ترمینال

۲. تلاش برای ایجاد پردازش جدید در ترمینال دیگر (مثلاً اجرای **ls** یا **ps**)

۳. مشاهده اینکه سیستم نمی‌تواند پردازش جدیدی ایجاد کند

۴. استفاده از اسکریپت **stop\_fork\_bomb.sh** یا دستورات **kill** برای متوقف کردن Bomb Fork

نتایج: **RESULT\_TODO**

## ۳ سوال ۳: نمایش و تحلیل کارایی برنامه

بهینه‌سازی عملکرد نرم‌افزار همواره به عنوان یکی از مراحل حیاتی در توسعه برنامه‌های کاربردی مطرح بوده است. در دنیای امروز که نیاز به پردازش داده‌های حجیم و اجرای محاسبات پیچیده روزبه‌روز در حال افزایش است، شناسایی گلوگاه‌های عملکردی و رفع آن‌ها نه تنها باعث بهبود سرعت اجرا می‌شود، بلکه مصرف منابع سیستم را نیز بهینه می‌کند.

### ۱.۳ ابزار perf در لینوکس

**perf** یک ابزار قدرتمند برای تحلیل عملکرد در لینوکس است که امکان جمع‌آوری داده‌های عملکردی در سطح هسته سیستم‌عامل و سخت‌افزار را فراهم می‌سازد.

ویژگی‌های اصلی **perf**:

- نمونه‌برداری از **CPU**: جمع‌آوری اطلاعات درباره توابعی که بیشترین زمان **CPU** را مصرف می‌کنند

- تحلیل رویدادهای سخت‌افزاری: دسترسی به Unit Monitoring Performance (PMU) برای تحلیل mispredictions branch misses، cache و غیره
- تحلیل stack call: امکان مشاهده زنجیره فراخوانی توابع
- تحلیل زمان واقعی: امکان مشاهده عملکرد برنامه در حین اجرا

دستورات مفید perf:

```
samples Record #
./program -g record perf
```

```
report Display #
report perf
```

```
display Interactive #
top perf
```

```
FlameGraph for output Save #
perf_script.out > script perf
```

RESULT\_TODO

## ۲.۳ برنامه C++ با بخش‌های نابینه

یک برنامه C++ پیچیده با بخش‌های نابینه در فایل performance\_test.cpp نوشته شده است که شامل موارد زیر است:

- محاسبه فاکتوریل به صورت بازگشتی بدون بهینه‌سازی
- جستجوی خطی در آرایه مرتب نشده
- مرتب‌سازی حبابی (Bubble Sort)
- محاسبات ریاضی پیچیده و تکراری
- کپی‌های غیرضروری از داده‌ها

این برنامه به گونه‌ای طراحی شده که حداقل ۱۰ ثانیه اجرای آن طول بکشد تا امکان تحلیل دقیق‌تر فراهم شود.  
کد برنامه: RESULT\_TODO

## ۳.۳ تحلیل عملکرد با perf

برای تحلیل عملکرد برنامه با perf، از اسکریپت run\_perf\_analysis.sh استفاده می‌شود:

```
run_perf_analysis.sh +x chmod
./run_perf_analysis.sh
```

این اسکریپت:

۱. برنامه را کامپایل می‌کند (با فلگ -g -O0 برای عدم بهینه‌سازی و شامل کردن اطلاعات debug)

۲. با perf record نمونه‌برداری انجام می‌دهد

۳. گزارش perf را تولید می‌کند

۴. خروجی را برای FlameGraph ذخیره می‌کند

نتایج تحلیل perf: RESULT\_TODO

## ۴.۳ FlameGraph

FlameGraph یک ابزار تجسم برای نمایش داده‌های عملکردی به صورت گرافیکی است. این ابزار داده‌های stack call را به صورت یک گراف افقی نمایش می‌دهد که در آن:

- عرض هر مستطیل نشان‌دهنده زمان صرف شده در آن تابع است
- ارتفاع نشان‌دهنده عمق stack call است
- رنگ‌ها برای تمایز بین توابع مختلف استفاده می‌شوند

نصب و استفاده:

```
https://github.com/brendangregg/FlameGraph.git clone git
FlameGraph cd
../flamegraph.svg > ./flamegraph.pl | -perf.pl./stackcollapse | script perf
```

یا با استفاده از اسکریپت generate\_flamegraph.sh:

```
generate_flamegraph.sh +x chmod
./generate_flamegraph.sh
```

RESULT\_TODO

## ۵.۳ تولید FlameGraph از خروجی perf

با استفاده از خروجی perf script می‌توان یک FlameGraph تولید کرد که بخش‌های نابینه برنامه را به وضوح نشان می‌دهد.

FlameGraph قبل از بهینه‌سازی: RESULT\_TODO

تحلیل FlameGraph و شناسایی بخش‌های نابینه: RESULT\_TODO

## ۶.۳ بهینه‌سازی و بهبود

بر اساس تحلیل FlameGraph، بخش‌های نابینه شناسایی و بهبود داده شدند:

- جایگزینی فاکتوریل بازگشتی با نسخه iterative یا استفاده از table lookup
- استفاده از جستجوی دودویی به جای جستجوی خطی (پس از مرتب‌سازی)
- جایگزینی مرتب‌سازی حبابی با الگوریتم‌های سریع‌تر مثل Sort Quick یا Sort Merge
- بهینه‌سازی محاسبات ریاضی با کاهش تکرارها و استفاده از tables lookup
- حذف کپی‌های غیرضروری با استفاده از reference یا semantics move

کد بهینه‌شده: RESULT\_TODO

## ۷.۳ نمایش نتایج بهینه‌سازی در FlameGraph

پس از بهینه‌سازی، دوباره برنامه اجرا شده و FlameGraph جدید تولید شد:  
FlameGraph بعد از بهینه‌سازی: RESULT\_TODO  
مقایسه نتایج:

- زمان اجرا قبل از بهینه‌سازی: RESULT\_TODO
- زمان اجرا بعد از بهینه‌سازی: RESULT\_TODO
- بهبود عملکرد: RESULT\_TODO

## ۸.۳ ابزار tracey

tracey یک تحلیل‌گر عملکرد بلادرنگ است که امکان مشاهده رفتار برنامه در حین اجرا را فراهم می‌کند.  
ویژگی‌های tracey:

- تحلیل بلادرنگ عملکرد
  - نمایش فراخوان‌های سیستمی
  - تحلیل I/O operations
  - نمایش activity network
- نصب و استفاده: RESULT\_TODO

## ۹.۳ تحلیل برنامه با tracey

برای تحلیل برنامه با tracey:  
./performance\_test tracey  
نتایج تحلیل: RESULT\_TODO

## ۴ سوال ۴

### ۱.۴ آ

مزایای معماری رابط واحد برای فایل و دستگاه از منظر انتزاع‌سازی و گسترش‌پذیری:

- **انتزاع‌سازی (Abstraction):** یک مدل شیء یکسان (فایل توصیف‌گر، عملیات open/read/write/close) برای منابع ناهمگن (فایل معمولی، سوکت، پایپ، دستگاه کاراکتری/بلوک) ارائه می‌شود؛ ساده‌سازی API برای برنامه‌نویس و کاهش کد شاخه‌ای.
- **گسترش‌پذیری (Extensibility):** افزودن نوع منبع جدید تنها با پیاده‌سازی همان واسط ممکن است (درایور جدید، سیستم فایل جدید) بدون تغییر برنامه‌های موجود.

## ۲.۴ ب

ioctl فراخوان سیستمی برای انجام عملیات کنترل/پیکربندی خاصِ دستگاه یا فایل است که در مدل ساده read/write قابل بیان نیست.

- **کارکرد:** انتقال فرمان‌های گیریکپارچه با کدهای cmd و پارامترهای in/out به درایور (مثلاً تنظیم baudrate پورت سریال، گرفتن اندازه ترمینال، دسترسی به ویژگی‌های اختصاصی).
- **رفع محدودیت:** ناتوانی POSIX در بیان عملیات غیرجریانی/غیرخطی (تنظیمات، out-of-band control) را دور می‌زند.
- **محدودیت‌ها:** ناهمگونی cmd ها، عدم پرتابل بودن ساختارها، نیاز به تطبیق نسخه، و سخت‌تر شدن بررسی‌های ایمنی type .

## ۳.۴ ج

در معماری میکرو هسته، همین رابط از طریق پیام‌رسانی به سرورهای فایل/دستگاه در فضای کاربر انجام می‌شود. مقایسه با هسته یکپارچه:

- **کارایی (Performance):** میکرو هسته دچار سربار پیام، صف‌بندی و رفت و برگشت‌های زمینه بین فرآیندهاست (IPC + سوئیچ کانتکست). هسته یکپارچه مسیر کوتاه‌تری دارد. با این حال، IPC سریع، اشتراک حافظه و zero-copy می‌تواند فاصله را کم کند.
- **قابلیت اطمینان (Reliability):** خرابی سرور در فضای کاربر به جای کرش کل سیستم، قابل بازیابی/ری‌استارت است. در هسته یکپارچه، نقص در درایور می‌تواند panic ایجاد کند.
- **امنیت (Security):** اصل حداقل امتیاز؛ سطح حمله کوچک‌تر در هسته؛ اعمال سیاست‌ها از طریق کانال پیام و قابلیت‌ها. در هسته یکپارچه، با وجود LSM، سطح امتیاز درایورها بالاتر و مرزها ضعیف‌تر است.

## ۴.۴ د

یک روش بهینه‌سازی مؤثر برای کاهش سربار پیام‌رسانی در معماری میکرو هسته، استفاده از Shared Memory + Zero-Copy است.

در این روش، بافرهای داده به صورت مستقیم بین فرآیند کلاینت و سرور فایل/دستگاه در فضای کاربر به اشتراک گذاشته می‌شوند. به جای کپی کردن داده‌ها در هر فراخوان سیستمی، فقط پیام‌های کنترل کوچک (مثل فلگ شروع/پایان، آدرس بافر مشترک، طول داده) از طریق IPC ارسال می‌شوند. کلاینت داده را مستقیماً در بافر مشترک می‌نویسد و سرور از همان بافر می‌خواند بدون نیاز به کپی اضافی. این روش چند مزیت دارد: کاهش تعداد کپی داده (از چند بار به صفر)، کاهش تعداد فراخوان‌های سیستمی، و کاهش تأخیر. جداسازی همچنان حفظ می‌شود چون دسترسی به بافر مشترک با کنترل دسترسی و قابلیت‌ها مدیریت می‌شود و فقط فرآیندهای مجاز می‌توانند به بافر دسترسی داشته باشند. این روش در سیستم‌هایی مثل Minix3 و QNX با موفقیت استفاده شده است.